

---

---

# ECE 471: Embedded Systems

**Date:**

**Final Project Write-up**

**Author:**

May 24, 2026

Nolan R. H. Gagnon

---

---

## 1 Introduction

This document describes the design process and functionality of an LED lighting system with color sensing capabilities. The system can perform various light shows (e.g., cycling through the colors of the rainbow and bouncing a strip of color back and forth). More light shows can easily be programmed for the system, since the code includes a library of high-level functions that present a simple interface to the RGB LED strip (the library was written in C by the author of this document). In addition to an RGB LED strip, the system has a color sensor that can detect the color of an object placed in front of it. If the user of the system wishes, the color of the RGB LED strip can be changed via the color sensor, by placing an object in front of it. In the current version of the software, red, green, and blue can be reliably detected by the system. Other colors, such as yellow or purple, are more difficult to process in the software. A more advanced color processing library is a work in progress. A picture of the current version of the system is provided below in Figure 1.

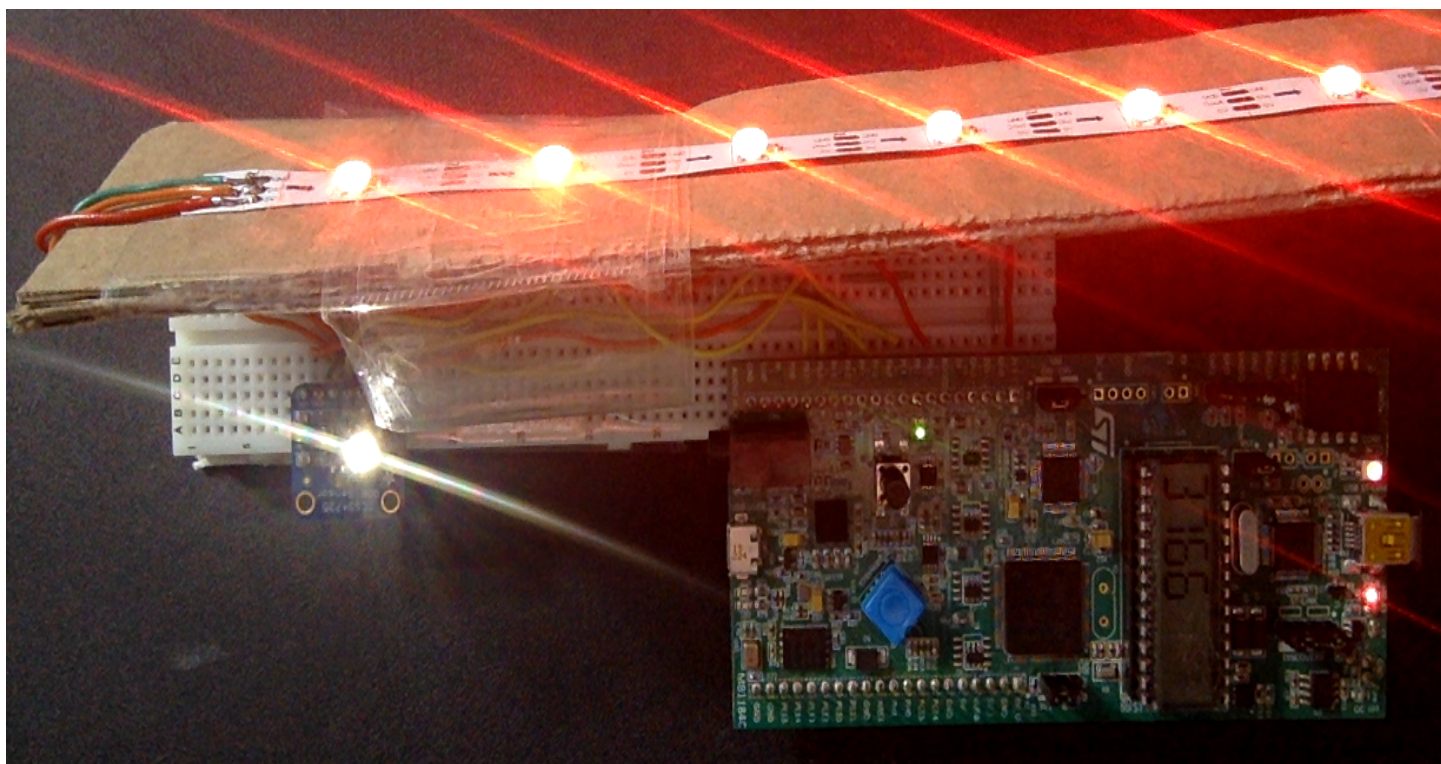


Figure 1: LED Lighting System

The small blue board is the color sensor, and the large green board is the STM32L476G Discovery Board, which is running the system software.

## 2 Hardware

The embedded board used for this project is the STM32L476G Discovery Board. It has many features, such as multiple I<sup>2</sup>C buses, multiple advanced timers that allow for the creation of PWM waveforms, a DMA (direct memory access) matrix, and excellent support for interrupt handling. The features mentioned make it an ideal board for this project. The board has an ARM Cortex-M4 processor; two internal SRAM chips with capacities of 96 KB and 32 KB, respectively; a 1 MB flash memory chip; and a maximum clock frequency of 80 MHz (which can be achieved using PLL hardware). The system is currently a bare-metal system (i.e., it has no operating system). At the moment an OS is not required, and could potentially interfere with the timing of the PWM waveforms being generated to control the RGB LED strip.

The color sensor is the TCS34725 Color Light-To-Digital Converter with IR Filter by Texas Advanced Optoelectronic Solutions. A close-up image of the sensor on a board from Adafruit can be seen below in Figure 2.



Figure 2: TCS34725 Board

The sensor is in the center of the board in Figure 2, and to its left is a white LED for illuminating objects. It functions as an ADC (analog to digital converter) capable of detecting red, green, blue, and clear light individually. The board can be programmed to generate an interrupt after every A/D conversion, which allows for more efficient code on the STM32L476G board. The light intensity threshold at which it will make a conversion can also be programmed. This allows the board to function properly in both dim and well-lit environments. Immediately after a conversion is completed, the INT line (interrupt line) on the TCS34725 will be pulled low to signal to the STM32L476G that it has data ready for reading (note that the line is connected to a 3.3 V rail by an external 10 k $\Omega$  pull-up resistor). The software on the STM32L476G will then enter an

interrupt handler and perform an I<sup>2</sup>C read of the digital color data. A sample I<sup>2</sup>C transaction between the TCS34725 and the STM32L476G can be seen in Figure 3 below. The yellow signal is on the SDA line and the blue signal is on the SCL line. This particular transaction segment is performed when the LED lighting system starts up and the TCS34725 settings are initialized. This waveform was captured using the Digilent Analog Discovery 2 and WaveForms 2015.

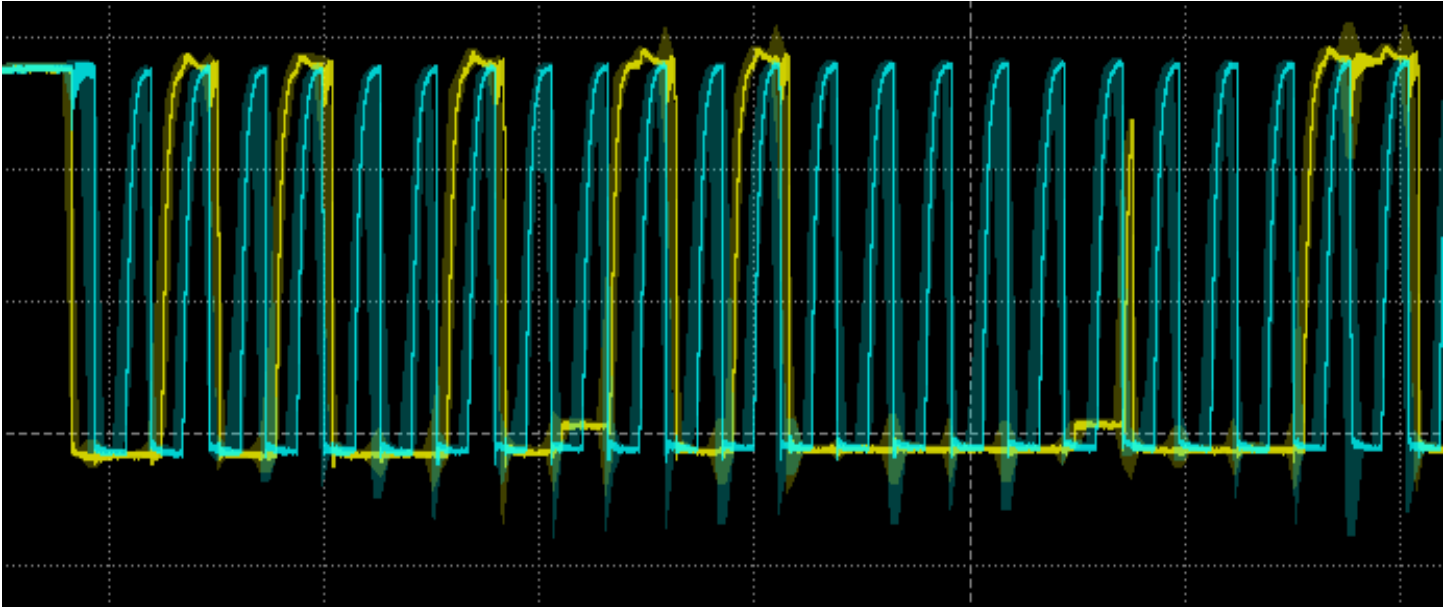


Figure 3: I<sup>2</sup>C Transaction in WaveForms 2015

The LED strip consists of 30 WS2812 24-bit RGB LEDs. When sending data to an LED, the first 8 bits are for the amount of green desired, the second 8 bits are for the amount of red desired, and the last 8 bits are for the amount of blue desired. The signals for sending “1 bits” and “0 bits” can be seen in Figure 4 below.

#### Sequence chart:

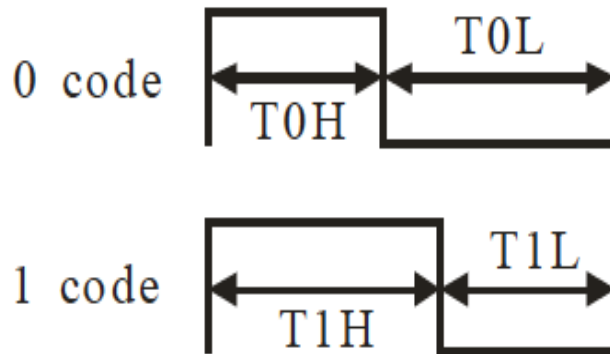


Figure 4: Sending 0 and 1 bits to WS2812 LEDs

T0H must have a width of  $0.35 \mu\text{s} \pm 150 \text{ ns}$ ; T0L must have a width of  $0.8 \mu\text{s} \pm 150 \text{ ns}$ ; T1H must have a width of  $0.7 \mu\text{s} \pm 150 \text{ ns}$ ; and T1L must have a width of  $0.6 \mu\text{s} \pm 150 \text{ ns}$ . With a chain of 30 WS2812 LEDs, all  $30 \times 24 = 720$  data bits can be sent at once. Each LED in the strip will only read the data it needs and pass the rest on to the remaining LEDs. A

PWM waveform generated by one of the STM32L476G's timers is used as the means of sending the color data to the LEDs. The timer has a counter that counts up to a specified value stored in its ARR (auto-reload) register. Whenever the counter reaches a programmed CCR (capture-and-compare) value, the polarity of the PWM waveform is inverted. A diagram of this process can be seen in Figure 5 below. This diagram is directly from the STM32L476G's reference manual.

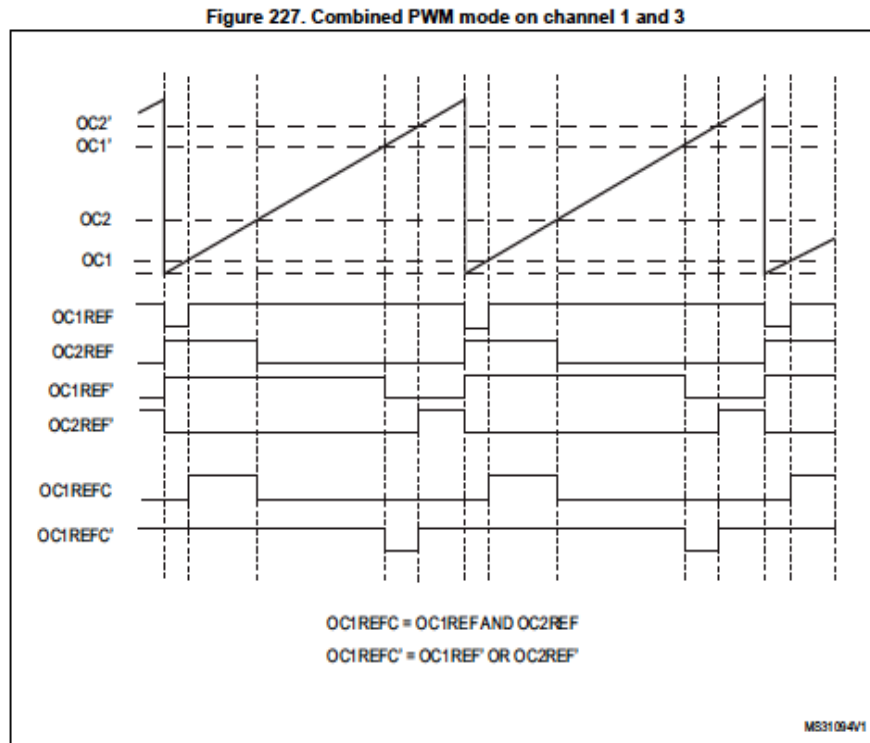


Figure 5: Generating a PWM Waveform

The triangle wave is the timer's counter, and the waveforms below it are various PWM outputs. The vertical dotted lines indicate CCR values. Since the WS2812 LEDs are expecting very specific waveforms (with a low tolerance for deviations), it is necessary to change the CCR value of the timer using DMA. Once the DMA is set-up, software isn't involved in changing the CCR values. Everything is done in hardware, which makes the process very fast. A segment of the PWM waveform sent to the LED strip when the system is performing its color bounce light show is shown below.

The data sheets for all hardware are in located in the folder called "Datasheets".

The power consumption for this system could potentially be problematic if enough LEDs were used. In order to optimize the power usage, however, the software turns off the PWM output whenever it is not needed. If the color of an LED is set and not changing, there is no reason to continue sending a PWM waveform to it. So in situations where no color transitions occur, the PWM is off.

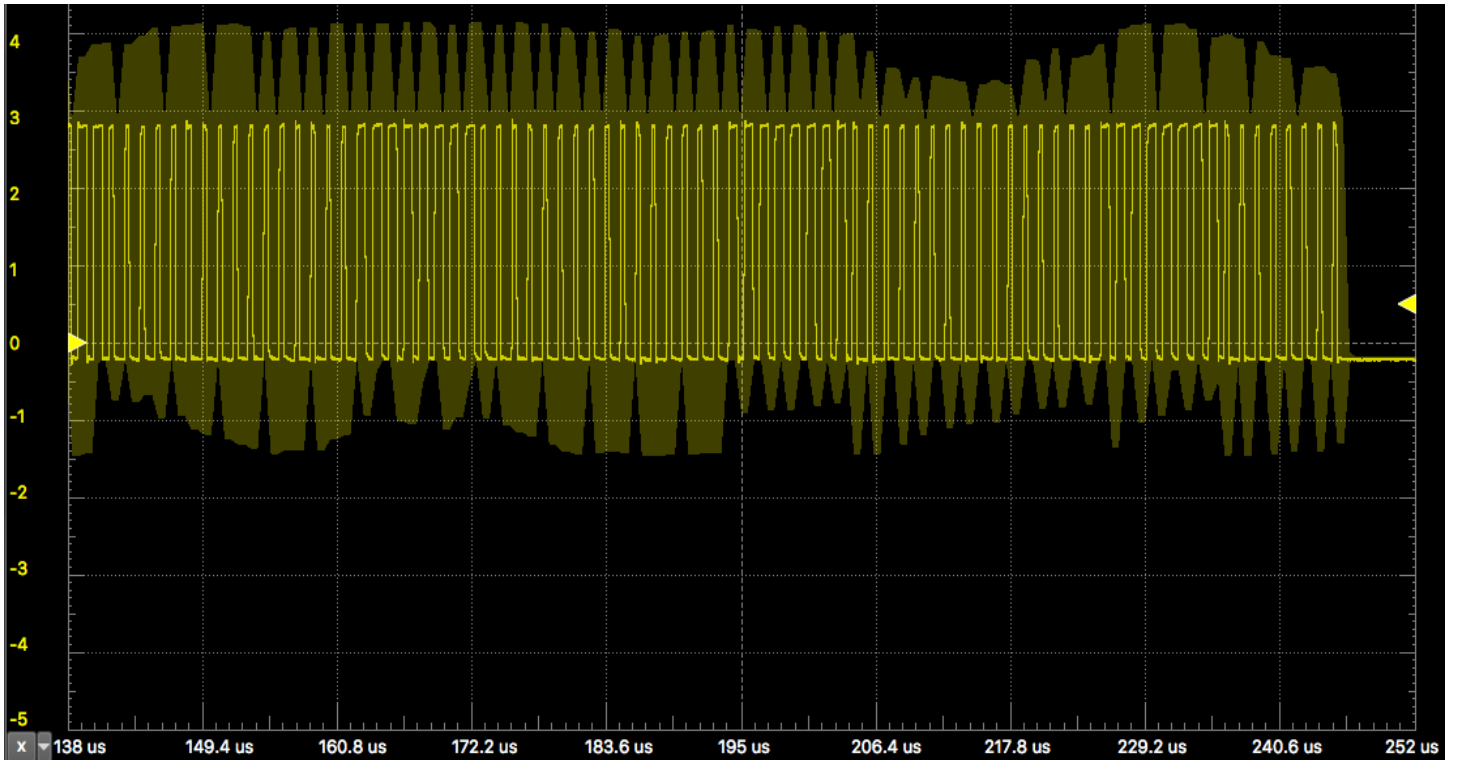


Figure 6: PWM Waveform During Light Show

### 3 Software

The system’s software is written in C. This was the best language choice for the system, since the software was written entirely from scratch, without any non-standard libraries (by standard libraries, I mean, e.g., `stdlib`). C is the best language for writing low-level code that interfaces with hardware directly (other than maybe assembly). If Assembly had been used for this project, there would have been no tangible benefits. C provided the right amount of control over the hardware, and writing the software in assembly would have taken a very long time, since the software system comprises many different files.

This is a firm real-time system. Hence the use of DMA to change the timer’s CCR values. If the CCR values are not changed by the deadline, the colors on the LED strip will not be correct.

As for security issues, there currently aren’t any. With the hardware the system has now, there is no way for it to connect to the Internet. The only way someone could tamper with the system is if they had physical access to it. However, there is not much to gain from tampering with it.

Besides standard libraries (such as `stdlib.h`, `perror.h`, `stdio.h`, etc.) and the `stm32l476xx.h` file, all of the code was written from scratch by me.

### 4 Related Work

There is a project similar to mine on Adafruit (see <https://learn.adafruit.com/adafruit-color-sensors>). However, my project requires a lot more knowledge about embedded systems than theirs does, since everything in my project was written from

scratch, based on the reference manual for the STM32L476G board and the data sheets for the LEDs and color sensor.

## 5 Conclusion

I worked alone on this project.

The biggest challenge for this project was processing the color data from the TCS34725 and sending it to the LED strip. The raw data output from the TCS34725 wasn't good enough to be sent directly to the LED strip. This was partially due to light noise going into the sensor before an A/D conversion. The raw data had to be analyzed in order to detect which color was most prominent, which was second-most prominent, and which was least most prominent. These colors then had to be enhanced (or un-enhanced) before they could be sent to the LED strip (otherwise, the color on the LED strip would not have been distinguishable).

Another issue was getting I<sup>2</sup>C to work from scratch. The STM32L476G reference manual is very long and complicated, so it took some time to understand how to properly set-up the I<sup>2</sup>C registers.

For future work, I would like to complete the color processing library so that more colors can be properly detected. I would also like to put the entire system into the shape of a usable lamp and turn it into an actual product.